주제: AI Agent (Single Agent) 구축

[목차]

1. AI Agent의 개념과 LLM과의 차이
2. 에이전트의 구성 요소
3. 에이전트의 동작 방식 (ReAct 루프)
4. 도구 사용과 Function Calling
5. 메모리와 상태 관리
6. 프롬프트 설계와 출력 제어
7. Single Agent 구축 단계와 프레임워크
8. 평가, 안전장치, 한계

1. AI Agent의 개념과 LLM과의 차이

LLM은 입력을 받으면 텍스트를 한 번 생성하고 끝난다.
반면 AI Agent는 목표가 주어지면 스스로 필요한 단계를 판단하고, 외부 도구를 사용하고, 결과를 확인하면서 목표를 달성할 때까지 반복한다.

예를 들어 "내일 서울 날씨 보고 우산 챙겨야 하는지 알려줘"라는 요청에서

- LLM은 학습한 지식만으로 답하므로 내일 날씨를 알 수 없다.
- Agent는 날씨 API를 호출해 실제 예보를 받아오고, 그 결과를 바탕으로 답한다.

Single Agent는 하나의 에이전트가 모든 판단과 도구 사용을 맡는 구조이다.
여러 에이전트가 역할을 나눠 협업하는 Multi-Agent와 구분된다.

[용어 설명]

- AI Agent: 목표를 달성하기 위해 스스로 판단하고 도구를 사용하는 LLM 기반 시스템이다.
- Single Agent: 에이전트가 하나뿐인 구조이다.
- Multi-Agent: 여러 에이전트가 역할을 나눠 협업하는 구조이다.

2. 에이전트의 구성 요소

Single Agent는 보통 다음 네 가지로 이루어진다.

- LLM (두뇌 역할의 모델): 상황을 이해하고 다음에 무엇을 할지 결정한다.
- 프롬프트: 에이전트의 역할, 규칙, 말투, 해야 할 일을 정해둔 지시문이다.
- 도구 (Tools): 에이전트가 호출할 수 있는 외부 기능이다. 검색, 계산기, 데이터베이스 조회, API 호출 등이 있다.
- 메모리: 이전 대화와 작업 결과를 기억하는 저장 공간이다.

LLM은 문장만 생성할 수 있으므로, 실제 행동(검색, 파일 읽기 등)은 도구가 대신 수행하고 그 결과를 LLM에게 돌려준다.

[용어 설명]

- 도구 (Tool): LLM이 호출할 수 있도록 만들어둔 함수나 API이다.
- 시스템 프롬프트: 대화 시작 전에 에이전트에게 주는 기본 지시문이다.

3. 에이전트의 동작 방식 (ReAct 루프)

가장 널리 쓰이는 방식은 ReAct이다. 아래 세 단계를 목표를 달성할 때까지 반복한다.

1. Thought (생각): 지금 무엇이 필요한지 판단한다.
2. Action (행동): 필요한 도구를 선택해 호출한다.
3. Observation (관찰): 도구가 돌려준 결과를 확인한다.

예시: "부산에서 2시간 안에 갈 수 있는 바다 사진 같은 여행지를 찾아줘"

- 생각: 먼저 사진과 비슷한 장소를 검색해야 한다.
- 행동: 장소 검색 도구를 호출한다.
- 관찰: 후보 5곳이 반환되었다.

- 생각: 이동시간을 확인해야 한다.
- 행동: 이동시간 조회 도구를 호출한다.
- 관찰: 2곳이 2시간 이내이다.

- 생각: 충분한 정보가 모였다. 답변을 작성한다.
- 루프가 끝나는 조건은 "최종 답변을 만들 준비가 됨"이다. 무한 반복을 막기 위해 최대 반복 횟수를 정해둔다.

[용어 설명]

- ReAct (Reasoning + Acting): 추론과 행동을 번갈아 수행하는 에이전트 방식이다.
- 루프 (Loop): 같은 과정을 조건이 만족될 때까지 반복하는 구조이다.

4. 도구 사용과 Function Calling

Function Calling은 LLM이 "이 도구를 이 입력값으로 호출하겠다"를 정해진 형식(JSON)으로 출력하게 하는 기능이다. 실제 호출은 우리가 작성한 코드가 수행한다.

동작 순서는 다음과 같다.

1. 도구의 이름, 설명, 입력 항목을 LLM에게 미리 알려준다.
2. LLM이 질문을 보고 호출할 도구와 입력값을 JSON으로 출력한다.
3. 코드가 그 도구를 실제로 실행한다.
4. 실행 결과를 LLM에게 다시 전달한다.
5. LLM이 결과를 바탕으로 다음 행동이나 최종 답변을 정한다.

예시 (날씨 도구):

- 도구 설명: "특정 도시의 날씨를 조회한다. 입력: city, date"
- LLM 출력: {"tool": "get_weather", "args": {"city": "서울", "date": "내일"}}

도구 설명을 명확하게 쓰는 것이 중요하다. LLM은 설명을 보고 도구를 고르기 때문에, 설명이 모호하면 엉뚱한 도구를 호출하거나 입력값을 잘못 넣는다.

[용어 설명]

- JSON: 데이터를 {"이름": 값} 형태로 표현하는 텍스트 형식이다.
- API: 프로그램끼리 기능을 주고받는 규칙이다. 날씨 API는 날씨 정보를 요청하면 돌려준다.
- 인자 (Argument): 함수를 호출할 때 넘기는 입력값이다.

5. 메모리와 상태 관리

LLM은 기본적으로 이전 대화를 기억하지 못한다. 그래서 필요한 내용을 매번 입력에 포함해야 하고, 이를 관리하는 것이 메모리이다.

- 단기 메모리: 현재 대화의 메시지와 도구 호출 결과이다. 대화 기록 자체를 매번 LLM에 함께 넣는 방식으로 구현한다.
- 장기 메모리: 대화가 끝난 뒤에도 남겨야 하는 정보이다. 사용자 선호, 이전 작업 결과 등을 **데이터베이스**나 **벡터 저장소**에 저장하고 필요할 때 검색해 가져온다. (RAG와 같은 원리이다.)

한 번에 LLM에 넣을 수 있는 길이에는 한계가 있어서, 대화가 길어지면 오래된 내용은 요약하거나 삭제한다.

[용어 설명]

- 컨텍스트 윈도우: LLM이 한 번에 처리할 수 있는 입력의 최대 길이이다.
- 상태 (State): 에이전트가 작업하는 중 현재까지 모은 정보(조건, 중간 결과 등)이다.

6. 프롬프트 설계와 출력 제어

에이전트의 품질은 **프롬프트**에 크게 좌우된다. 시스템 프롬프트에는 보통 다음을 포함한다.

- 역할: “너는 국내 여행지 추천 도우미이다.”
- 사용 가능한 도구와 사용 기준: “이동시간은 반드시 이동시간 도구로 확인한다.”
- 규칙과 금지 사항: “도구 결과에 없는 정보는 지어내지 않는다.”
- 출력 형식: “답변은 장소명, 이유, 이동시간 순으로 쓴다.”

프로그램이 에이전트의 출력을 이어서 처리해야 할 때는 구조화 출력을 사용한다. 자유로운 문장 대신 정해진 JSON 형식으로 답하게 하는 것이다. 예를 들어 사용자 문장에서 조건을 뽑을 때 {"origin": "서울", "max_travel_minutes": 120}처럼 필드를 고정하면, 이후 코드에서 바로 사용할 수 있다.

[용어 설명]

- 구조화 출력 (Structured Output): 정해진 형식과 필드로 출력하게 하는 방식이다.
- 스키마 (Schema): 데이터에 어떤 항목이 있고 각각 어떤 형태인지 정해둔 틀이다.

7. Single Agent 구축 단계와 프레임워크

구축은 다음 순서로 진행한다.

1. 목표 정의: 에이전트가 해결할 문제와 성공 기준을 정한다.
2. 도구 설계: 필요한 도구를 정하고, 각 도구의 이름·설명·입력·출력을 정의한다.
3. 프롬프트 작성: 역할, 규칙, 출력 형식을 작성한다.
4. 루프 구현: 생각 → 도구 호출 → 결과 반영을 반복하고 종료 조건과 최대 반복 횟수를 넣는다.
5. 메모리 연결: 대화 기록과 필요한 장기 정보를 연결한다.
6. 테스트와 개선: 다양한 질문으로 실행해보고 실패 사례를 보완한다.

직접 구현할 수도 있고 프레임워크를 사용할 수도 있다.

- LangChain / LangGraph: 에이전트, 도구, 메모리를 쉽게 연결해주는 대표적인 프레임워크이다. LangGraph는 단계를 그래프 형태로 정의해 흐름을 세밀하게 제어한다.
- LLM 제공사 SDK: Function Calling 기능을 직접 호출해 구현한다. 구조가 단순하고 동작을 이해하기 쉽다.

[용어 설명]

- 프레임워크: 자주 쓰는 기능을 미리 만들어 제공하는 개발 도구 모음이다.
- SDK: 특정 서비스를 코드에서 쉽게 사용하도록 제공하는 라이브러리이다.

8. 평가, 안전장치, 한계

평가는 최종 답변만 보지 않고 과정도 확인한다.

- 작업 성공률: 목표를 제대로 달성했는가.
- 도구 호출 정확도: 올바른 도구를 올바른 입력값으로 호출했는가.
- 효율성: 불필요한 호출 없이 적은 단계와 시간으로 끝냈는가.
- 근거 충실도: 답변이 도구 결과에 근거하는가.

안전장치도 필요하다.

- 최대 반복 횟수 제한: 무한 루프를 막는다.
- 호출 상한과 비용 제한: API 호출이 과도하게 늘어나는 것을 막는다.
- 입력값 검증: LLM이 만든 도구 입력값이 올바른 형식인지 실행 전에 확인한다.
- 권한 제한: 삭제, 결제처럼 되돌리기 어려운 작업은 사용자 확인을 거친다.
- 오류 처리: 도구 호출이 실패했을 때 재시도하거나 실패 사실을 알리게 한다.
- 로깅: 어떤 도구를 왜 호출했는지 기록해서 문제 원인을 추적한다.

한계도 알아야 한다.

- 도구를 잘못 고르거나 입력값을 틀릴 수 있다.
- 호출 횟수가 늘수록 응답이 느려지고 비용이 증가한다.
- 도구가 많아지고 작업이 복잡해지면 하나의 에이전트가 감당하기 어렵다. 이때 역할을 나누는 Multi-Agent를 고려한다.

[용어 설명]

- 로깅 (Logging): 실행 과정과 결과를 기록으로 남기는 것이다.
- 무한 루프: 종료 조건에 도달하지 못하고 계속 반복되는 상태이다.